name argument, as you'll want to reference this view in your HTML. As you haven't written the view for the form partial, clicking the Edit link won't work, but you can see the main page. Visit http://localhost:8000/bands/room_editor/1/ to see the rooms associated with the CGBG venue.

The next step is to add the view that handles the form to edit a room. Similarly to the form views covered in chapter 7, this view is responsible for both displaying the form and handling the submit. For this view, the HTML rendered is a partial segment in both cases, but otherwise, the idea is the same. The form, in this case, is simple enough that you could handle the request.POST manually, but I'll stick with best practices and create a ModelForm, shown in the following listing.

Listing 12.15 A form for editing Room objects

```
# RiffMates/bands/forms.py
from django import forms
...
from bands.models import Room
...

RoomForm = forms.modelform_factory(Room, fields=["name",])
```

The view that handles the room form has both a GET and POST case. The GET renders the form with the given room data. For the POST, the form gets validated, and the submitted data gets saved, replacing the values in the associated room. In both cases, the output is a partial, so HTMX can either replace the display information with the form or the form with the display information. The following listing shows the view.

Listing 12.16 An HTMX view for handling a room-editing form

```
# RiffMates/bands/views.py
...
from bands.forms import RoomForm
...
@login_required
def edit_room_form(request, room_id):
    room = get_object_or_404(Room, id=room_id,
        venue__userprofile=request.user.userprofile)

    if request.method == "POST":
        form = RoomForm(request.POST, instance=room)
        if form.is_valid():
            form.save()
            data = {
                "room": room,
            }
            return render(request, "partials/show_room.html",
                data)
    else:
        form = RoomForm(instance=room)

    data = {
```

Editing data requires the user to be logged in.

Edit the Room object for the ID passed in.

Fetch the Room, and ensure it is associated with a venue that is associated with the user.

The view handles form submission through a POST method.

Validate, and then save the submitted room data.

After saving the changes, show the partial displaying the room info.

For the GET scenario, build the form based on the Room object.